Redis vs MongoDB for Trip Planning with GTFS Data

An application-based benchmark · Chimaroke Amadi · GoMyCode MSC checkpoint · 8 November 2026

I loaded two **real** public transport timetables into Redis and MongoDB: Big Blue Bus (Santa Monica, 123,699 scheduled stops) and TriMet (Portland, **3,407,790**). Then I ran the questions a trip planner really asks, like "next buses from this stop" and "how do I get from A to B", 2,000 times each on each database. I checked that both databases gave the same answers, and measured speed, memory, throughput and how the results change when the network is 28 times bigger. Finally, I built a small trip-planner web app that runs on either database.

Single queries (TriMet, median)	Whole trip plan (TriMet, median)	Peak throughput (TriMet)	Space for TriMet
2.5×	**2.13 vs 9.41 ms**	**11.0K vs 7.2K**	**395 MB vs 224 MB**
MongoDB time ÷ Redis time, on average	Redis vs MongoDB, about 29 database calls each	operations per second, Redis vs MongoDB	Redis in RAM vs MongoDB on disk

The short answer: Redis was faster on every query on the big network (on the small one, MongoDB won only the name search), and much faster for the full app, because a trip plan makes many small calls. But it keeps everything in RAM, and I had to design every "index" by hand. MongoDB needed only 57% as much space (on disk, versus Redis's RAM), and found the better index by itself. It is also easier to grow beyond one machine. For a large trip-planning service, I would keep the timetable in **MongoDB** and add **Redis as a cache** for the hottest answers (section 6).

1. Introduction to GTFS data

What is GTFS, and what does it contain?

GTFS (General Transit Feed Specification) is the standard format that transit agencies use to publish their schedules. It began in 2005 as a project between TriMet in Portland (one of my two test feeds) and Google, which is why it was first called the "Google Transit Feed Specification". Today thousands of agencies publish GTFS, and apps like Google Maps and Transit read it.

A GTFS feed is just a **ZIP file of CSV text files**. Each file is like a table, and they link to each other by ids:

File	What it holds	Big Blue Bus	TriMet
`stops.txt`	every stop: id, name, latitude, longitude	901	6,031
`routes.txt`	bus or train lines: short name ("7"), long name, type, colour	17	82
`trips.txt`	one journey of a vehicle along a route, with the **service** (days) it runs	3,277	67,957
`stop_times.txt`	when each trip arrives at and leaves each stop, in order. This is by far the biggest file	123,699	3,407,790
`calendar.txt`, `calendar_dates.txt`	which days each service runs, plus exceptions (holidays)	5 and 6 services run on the test day	
`shapes.txt`, `transfers.txt`, fares…	map lines, transfer rules, prices (optional files)	not used here	

For example, this real row from TriMet's `stop_times.txt` says that trip 18188725 leaves stop 12845 (Marquam Hill) at 05:30, and it is the 1st stop of that trip:

```
trip_id,arrival_time,departure_time,stop_id,stop_sequence,stop_headsign,...
18188725,05:30:00,05:30:00,12845,1,Marquam Hill,...
```

GTFS has a few traps that a trip planner must handle (my tests check all of them):

- **Times can go past 24:00.** A bus at 1:10 am that belongs to the evening's service is written `25:10:00`, so I store times as "seconds since midnight", not as clock times.
- **Not every trip runs every day.** A trip belongs to a *service*, and the calendar files say which days that service runs. TriMet's file holds 47 services (several schedule periods), and only **8.9%** of its timetable rows run on a given Tuesday. This turned out to matter a lot (section 6).
- **Loop routes** pass the same stop twice, so "the bus from A to B" must pick the right visit to A.

How can GTFS data be used for trip planning?

Almost every feature of a journey planner is a question about these files:

- **"Which stops are near me?"**: a geographic search on `stops.txt`.
- **"When is the next bus from this stop?"**: `stop_times` for one stop, sorted by time, keeping only the services that run today.
- **"Where does this bus go?"**: all stop_times of one trip, in order.
- **"How do I get from A to B?"**: trips that pass A and later B (direct), or a first trip to a stop where a second trip continues to B (one change). Walking between nearby stops makes the plan realistic.
- Plus maps (`shapes.txt`), fares, accessibility (`wheelchair_boarding`), and live delays from the companion format **GTFS-Realtime**.

2. Overview of Redis and MongoDB

What are they, and how do they differ from SQL databases?

A traditional SQL database (MySQL, PostgreSQL) stores data in **tables with a fixed schema**, links them with foreign keys and **joins**, and is queried with SQL. Redis and MongoDB are both "NoSQL", but they are very different from each other:

- **Redis** is an **in-memory key-value store**. Every piece of data is a value stored under a key, and the value can be a data structure: a string, hash, list, set, **sorted set**, geo index, stream… There are no tables, no joins and no query language. You call commands like `ZRANGE` or `GEOSEARCH` on one key at a time, so **you design the keys around the questions you will ask**.
- **MongoDB** is a **document database**. Data is stored as JSON-like documents (BSON) in collections, and documents can have different fields. It has a real **query language**, secondary indexes, a query planner, and an aggregation pipeline. That makes it the closer of the two to SQL, but without joins by default (`$lookup` exists) and without a fixed schema.

	Redis 8.10.2	**MongoDB 8.2.6**
Data model	keys → data structures (hash, sorted set, geo…)	collections of documents (BSON)
Where data lives	all in RAM; optional snapshots (RDB) or a log (AOF) on disk	on disk (WiredTiger, compressed), hot data cached in RAM
Querying	commands on one key; logic in the application	query language, indexes chosen by a query planner, aggregation
Geo	`GEOADD` / `GEOSEARCH` (a sorted set with geohash scores)	GeoJSON + `2dsphere` index, `$near`, `$geoNear`
Scaling out	Redis Cluster: 16,384 hash slots spread over nodes; replicas	replica sets; sharding by a shard key
Typical role	cache, sessions, leaderboards, queues, real-time counters	main application database

Key features

Redis **MongoDB**

- Very low latency: data in RAM, simple commands, and one main thread that runs commands one by one (so each command is atomic).
- Rich data structures: a **sorted set** keeps members ordered by a score, which is perfect for "departures ordered by time".
- Built-in geo commands, pipelining (many commands in one round trip), transactions, pub/sub, streams, key expiry (TTL).
- Persistence is optional: RDB snapshots and/or an append-only file.

- Flexible documents: a GTFS row maps naturally to a document, and related data can be embedded.
- Secondary and compound indexes, geo (`2dsphere`), text, and a **query planner** that picks the best index.
- Aggregation pipeline (grouping, `$geoNear`, `$lookup`), transactions, change streams.
- Durable by default (journal), compressed storage, replication and sharding built in.

3. Benchmarking criteria

What I measured, and how

Criterion	How I measured it
Correctness	Every one of the 12,000 test queries per feed ran on both databases and the answers were compared. A fast wrong answer is worthless, so I checked this first. I also wrote 20 tests with known answers (loops, after-midnight times, holidays).
Ingestion speed	Time to load the whole feed (and build MongoDB's indexes), median of 3 runs from an empty database.
Memory / storage	Redis `MEMORY STATS` ; MongoDB `dbStats` after an `fsync` (data size, on-disk size, index size).
Query latency	6 trip-planning queries, 2,000 different realistic inputs each (after 200 warm-up queries), median (p50), 95th and 99th percentiles.
Throughput	A realistic mix (35% departure boards, 20% nearby stops, 20% A → B…), 6,000 operations with 1, 8, 32 and 128 users at once.
Scalability	The same tests on a small (124K rows) and a large (3.4M rows) real network.
Whole application	Time to plan a full journey with my trip planner (about 30 database calls), for 300 random journeys per feed.
Ease of use	Code size, what I had to build by hand, and how each database reacted when I changed the data layout.

Why benchmark with a real application scenario?

Generic benchmarks ("how many GET/SET per second") don't tell you how your app will behave, because the speed depends on the **data model and the query pattern**. My own results show this three times:

- The "next departures" query was 3× slower in Redis on the bigger feed. The database was fine; my key design read 11 rows for every useful one. A generic benchmark would never show that (section 6).
- Single queries differ by fractions of a millisecond, but a real trip plan makes about 29 calls, so the gap becomes 4.4× for the whole request.
- Correctness problems (loop routes, holidays) only show up with real data.

So I used real feeds, real questions, and compared the answers, not just the speed.

4. Data ingestion and storage

How I ingested GTFS into each database

The same pipeline feeds both databases, so the comparison is fair: **download** the ZIP → **parse** the CSV files as a stream (the TriMet `stop_times.txt` has 3.4 million lines) → **transform** (times to seconds, skip invalid stops, copy

`service_id` and `route_id` next to each stop time) → **load** in big batches. Parsing took 14.2 s for TriMet and is not counted below.

Redis: keys designed for each question

```
stop:{id}          HASH  name, code, lat, lon
stops:geo          GEO   position of every stop
stops:names        ZSET  name words, for search
route:{id}         HASH  short/long name, colour
trip:{id}          HASH  route, service, headsign
trip:{id}:stops    ZSET  score = stop_sequence
dep:{stop_id}      ZSET  score = departure time
                         (member:
trip|seq|service|route)
```

Loading uses **pipelines** of 1,000 commands, and one variadic `ZADD` per sorted set, instead of one command per row.

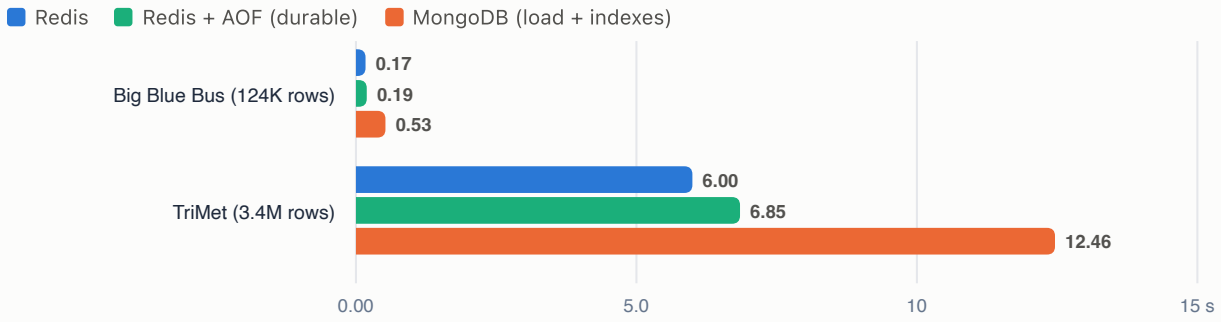
MongoDB: collections close to the files

```
stops       { _id, name, code, search: [...],
              location: GeoJSON Point }
routes      { _id, short_name, long_name, ... }
trips       { _id, route_id, service_id, ... }
stop_times  { trip_id, stop_id, seq, arr,
              dep, service_id, route_id }
indexes:
  stops       location (2dsphere), search
  stop_times  {stop_id, dep}, {trip_id, seq}
```

Loading uses unordered `insertMany` batches of 10,000, and indexes are built **after** the load.

Time to load each feed (median of 3 runs, seconds)

Lower is better. MongoDB includes building its 4 indexes (4.7 s of the 12.5 s for TriMet).

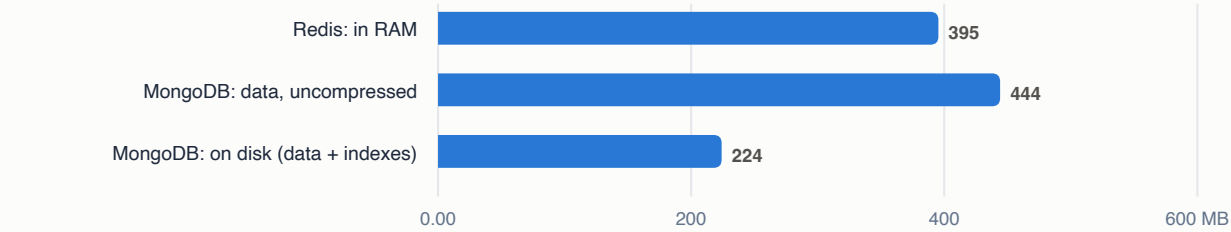


Redis loaded TriMet in **6.0 s** (568,168 stop times per second), MongoDB in **12.5 s**. Turning on Redis's append-only file (so the data survives a restart) only added 14%. Both are fast enough: a GTFS feed changes every few weeks, so loading it is a nightly job, not something users wait for.

Comparing the storage mechanisms

Space used by the TriMet feed (MB)

Redis keeps everything in RAM; MongoDB keeps compressed data and indexes on disk and caches the hot part in RAM.



- **Redis** keeps every key in **RAM**: 395 MB for TriMet, about 116 bytes per stop time (each stop time is stored twice: in its stop's departures and in its trip's stop list). Nothing is on disk unless you turn on RDB or AOF, and even then the disk copy is only for restarting. **The whole dataset must fit in memory.**
- **MongoDB** stores documents in WiredTiger B-trees on disk, **compressed** (snappy for data, prefix compression for indexes): 444 MB of documents take 98 MB on disk, plus 127 MB of indexes. Only the "working set" needs to be in RAM (its cache defaults to half of the RAM minus 1 GB), so a dataset can be **bigger than the memory**.
- A MongoDB document carries its field names, so the uncompressed data (444 MB) is actually bigger than Redis's. Compression on disk is what makes MongoDB the smaller one.

5. Query performance

How each database answers trip-planning questions

Redis has no query planner: each question is one or two commands on the keys I designed, and any filtering or combining happens in my Node.js code. **Pipelining** sends many commands in one network round trip. **MongoDB** receives a query, and its planner chooses an index, filters and sorts on the server, and returns only the result.

Typical queries (from my code)

Next 10 departures from a stop after 08:00

```
// Redis: read the stop's sorted set by time,
// skip services that don't run today
ZRANGE dep:13248 28800 +inf BYSCORE LIMIT 0 50

// MongoDB: index { stop_id: 1, dep: 1 }
db.stop_times.find({ stop_id: '13248',
    dep: { $gte: 28800 },
    service_id: { $in: todaysServices } })
  .sort({ dep: 1 }).limit(10)
```

Stops within 500 m of a point

```
// Redis: geo search, then 1 pipeline for names
GEOSEARCH stops:geo FROMLONLAT -122.68 45.52
  BYRADIUS 500 m ASC COUNT 10 WITHDIST

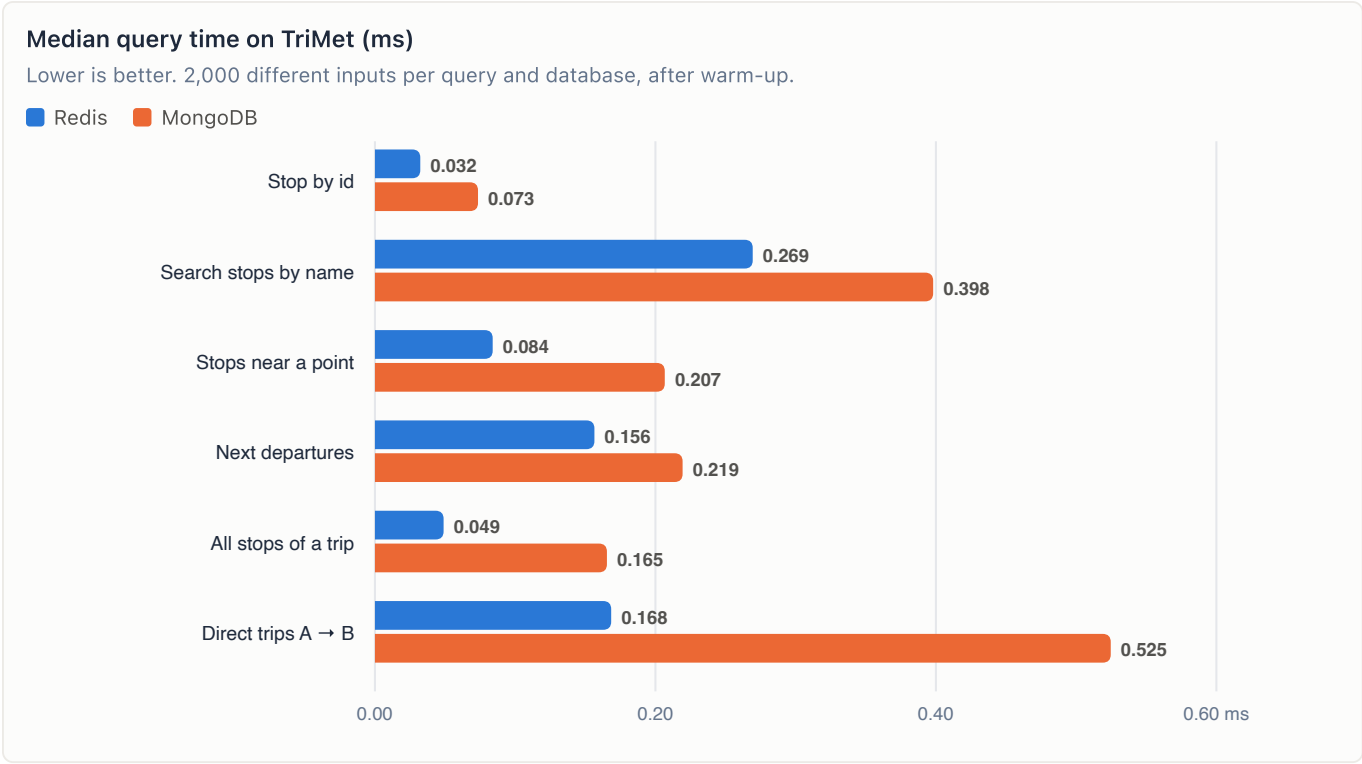
// MongoDB: 2dsphere index
db.stops.aggregate([{ $geoNear: {
  near: { type: 'Point',
          coordinates: [-122.68, 45.52] },
  maxDistance: 500, distanceField: 'distance',
  spherical: true } }, { $limit: 10 }])
```

Direct trips from A to B use the same 2 steps on both: (1) the departures from A in the next 2 hours, (2) for those trips, find B *later* in the trip: one Redis pipeline of `ZRANGEBYSCORE trip:{id}:stops`, or one MongoDB query `{ trip_id: { $in: [...] }, stop_id: B }` using the `{trip_id, seq}` index. The other queries are a stop by id (`HGETALL` vs `findOne`), a trip's stops (`ZRANGE` vs `find().sort({seq})`), and a name search ("sawt" finds "OLYMPIC WB & SAWTELLE NS": a `BYLEX` range in Redis, an anchored regex on an indexed array in MongoDB).

Are the answers the same?

Feed	Stop by id	Search stops by name	Stops near a point	Next departures	All stops of a trip	Direct trips A → B
Big Blue Bus	2,000	2,000	1,958 + 42*	1,999 + 1*	2,000	2,000
TriMet	2,000	2,000	1,956 + 44*	1,993 + 7*	2,000	2,000

Out of 24,000 compared queries, **0 were different**. The ones marked * are equivalent but not identical: two stops at almost the same distance came back in a different order (Redis stores positions as 52-bit geohashes and uses a slightly different Earth radius, so distances differ by a few centimetres), or two buses leaving in the same minute were listed in a different order. While building the benchmark, the comparison also caught a real bug in my first version: on loop routes, Redis listed the same bus twice and MongoDB could pick the wrong visit. I fixed it in both before measuring.



Query	Big Blue Bus (median ms)			TriMet (median ms)			TriMet 95th percentile (ms)	
	Redis	MongoDB	Mongo ÷ Redis	Redis	MongoDB	Mongo ÷ Redis	Redis	MongoDB
Stop by id	0.031	0.070	2.3×	0.032	0.073	2.3×	0.059	0.083
Search stops by name	0.275	0.231	0.8×	0.269	0.398	1.5×	2.12	2.10
Stops near a point	0.078	0.198	2.5×	0.084	0.207	2.5×	0.124	0.289
Next departures	0.048	0.117	2.4×	0.156	0.219	1.4×	0.173	0.268
All stops of a trip	0.042	0.150	3.6×	0.049	0.165	3.4×	0.072	0.320
Direct trips A → B	0.137	0.386	2.8×	0.168	0.525	3.1×	0.396	1.08

- **Redis was 2.5× faster on average** for the simple lookups (stop, nearby, trip, departures, A → B), and its 95th percentiles for those queries were at most 0.396 ms.
- **Name search was the closest race:** MongoDB won on the small feed (0.231 vs 0.275 ms), Redis on TriMet (0.269 vs 0.398 ms), and the 95th percentiles were the same (about 2.12 ms). For short prefixes like "stat" both fetch up to 1,000 matches and sort them, so that work, not the database, dominates.
- The **median of every query was under a millisecond** on both databases. For a single request, the user would never notice the difference; it matters when there are many calls per request or many users (next sections).

6. Scalability and efficiency

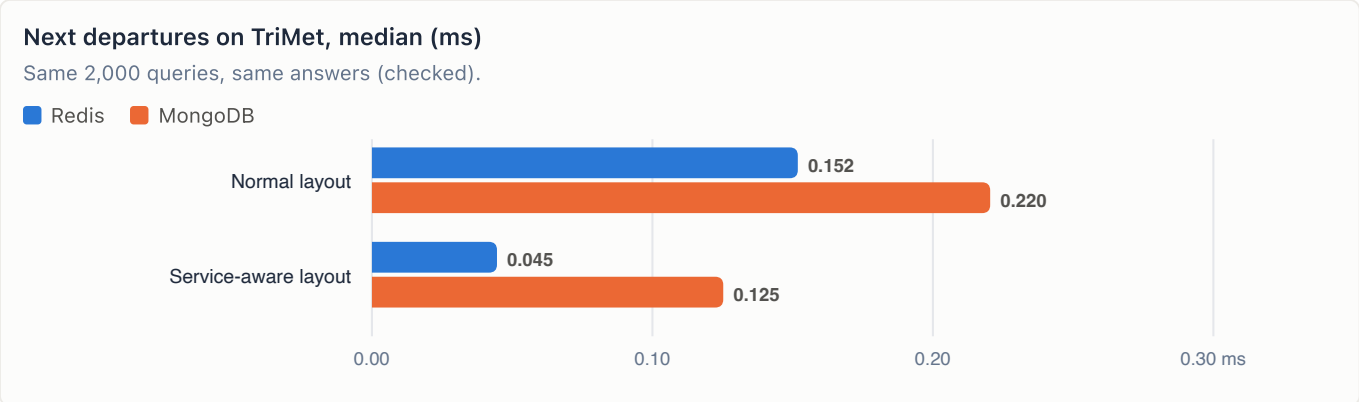
From a small to a large network

	Big Blue Bus	TriMet	Growth
Stop times	123,699	3,407,790	28×
Redis load time / RAM	0.17 s / 14 MB	6.0 s / 395 MB	35× / 28×
MongoDB load time / disk	0.53 s / 8.1 MB	12.5 s / 224 MB	24× / 28×
Stop by id, median (Redis / MongoDB)	0.031 / 0.070 ms	0.032 / 0.073 ms	1.0× / 1.0×
Stops near a point, median (Redis / MongoDB)	0.078 / 0.198 ms	0.084 / 0.207 ms	1.1× / 1.0×
All stops of a trip, median (Redis / MongoDB)	0.042 / 0.150 ms	0.049 / 0.165 ms	1.2× / 1.1×
Next departures, median (Redis / MongoDB)	0.048 / 0.117 ms	0.156 / 0.219 ms	3.2× / 1.9×
Direct trips A → B, median (Redis / MongoDB)	0.137 / 0.386 ms	0.168 / 0.525 ms	1.2× / 1.4×

Load time and space grew roughly in line with the data (both are linear). Thanks to indexes and key lookups, **most queries barely slowed down** with 28 times more data, on both databases: they read a few entries whatever the size. The exception was **next departures in Redis: 3.2× slower**.

The surprise: data layout matters more than the database

I looked into it. In TriMet's file only 8.9% of the timetable rows run on a given day, so my `dep:{stop}` sorted set mixes today's departures with those of 40 other service periods. To return 10 departures, Redis read them in chunks and skipped about 9 out of 10 in my code, with several round trips. MongoDB did the same filtering inside the server, in one round trip, which is why it slowed down less. I tried a **service-aware layout** on both: Redis keys `dep:{stop}:{service}` (read the running services in one pipeline and merge), and a MongoDB index `{stop_id, service_id, dep}`.

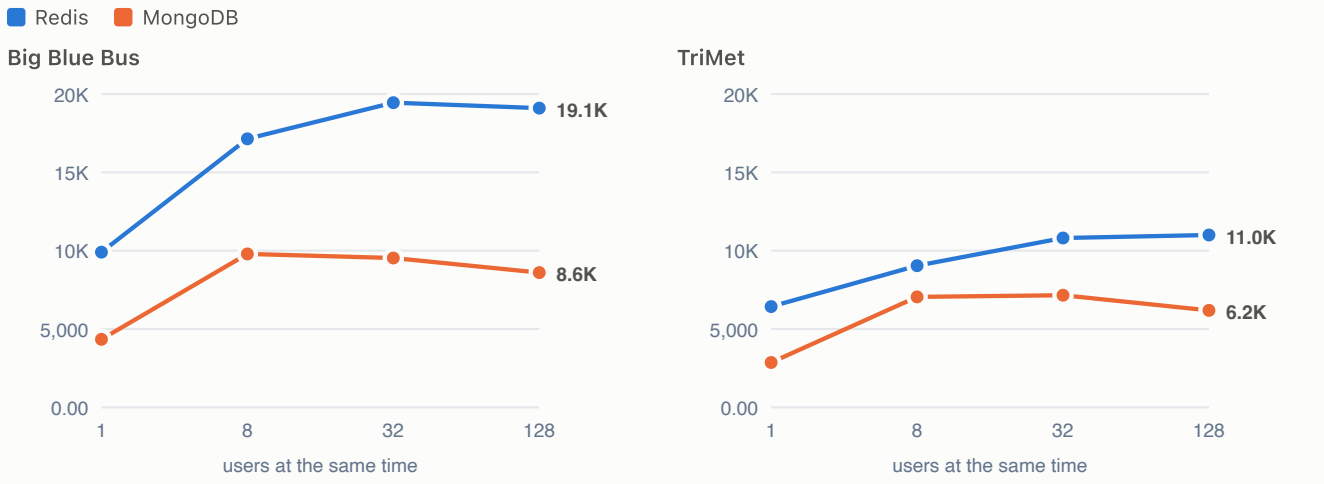


- Redis: **0.152 → 0.045 ms** (3.4× faster; 95th percentile 0.170 → 0.049 ms), back to the small-feed speed. Cost: 68,648 more keys and 104 MB more RAM when both layouts are kept.
- MongoDB: **0.220 → 0.125 ms**. `explain()` on the busiest stop shows why: 49 index entries read to return 10 before, 11 after. Cost: 59 MB of index.
- **Ease of use:** once the index existed, MongoDB's query planner **chose it by itself**, with no code change (my first measurement was wrong because of this, and I had to force the old index to compare). In Redis I had to change both the keys and the query code.

Throughput: many users at once

Operations per second with 1 to 128 users at the same time

Higher is better. Realistic mix of trip-planning queries; client and database on the same laptop.



Users at the same time	1	8	32	128
Big Blue Bus, Redis	9,905 ops/s p95 0.375 ms	17,139 ops/s p95 1.43 ms	19,446 ops/s p95 3.63 ms	19,098 ops/s p95 10.6 ms
Big Blue Bus, MongoDB	4,341 ops/s p95 0.632 ms	9,793 ops/s p95 2.10 ms	9,532 ops/s p95 7.43 ms	8,602 ops/s p95 25.7 ms
TriMet, Redis	6,432 ops/s p95 0.370 ms	9,041 ops/s p95 2.47 ms	10,809 ops/s p95 5.82 ms	11,001 ops/s p95 18.8 ms
TriMet, MongoDB	2,864 ops/s p95 0.895 ms	7,050 ops/s p95 3.83 ms	7,156 ops/s p95 10.0 ms	6,189 ops/s p95 38.0 ms

Redis handled about **1.5×** the peak throughput of MongoDB on TriMet and kept lower tail latency. Both flatten after about 8–32 users: here the benchmark client (one Node.js process on the same 11-core laptop) and Redis's single command thread become the limit. With 128 users, MongoDB's throughput dropped and its 95th percentile rose to 38.0 ms, against 18.8 ms for Redis.

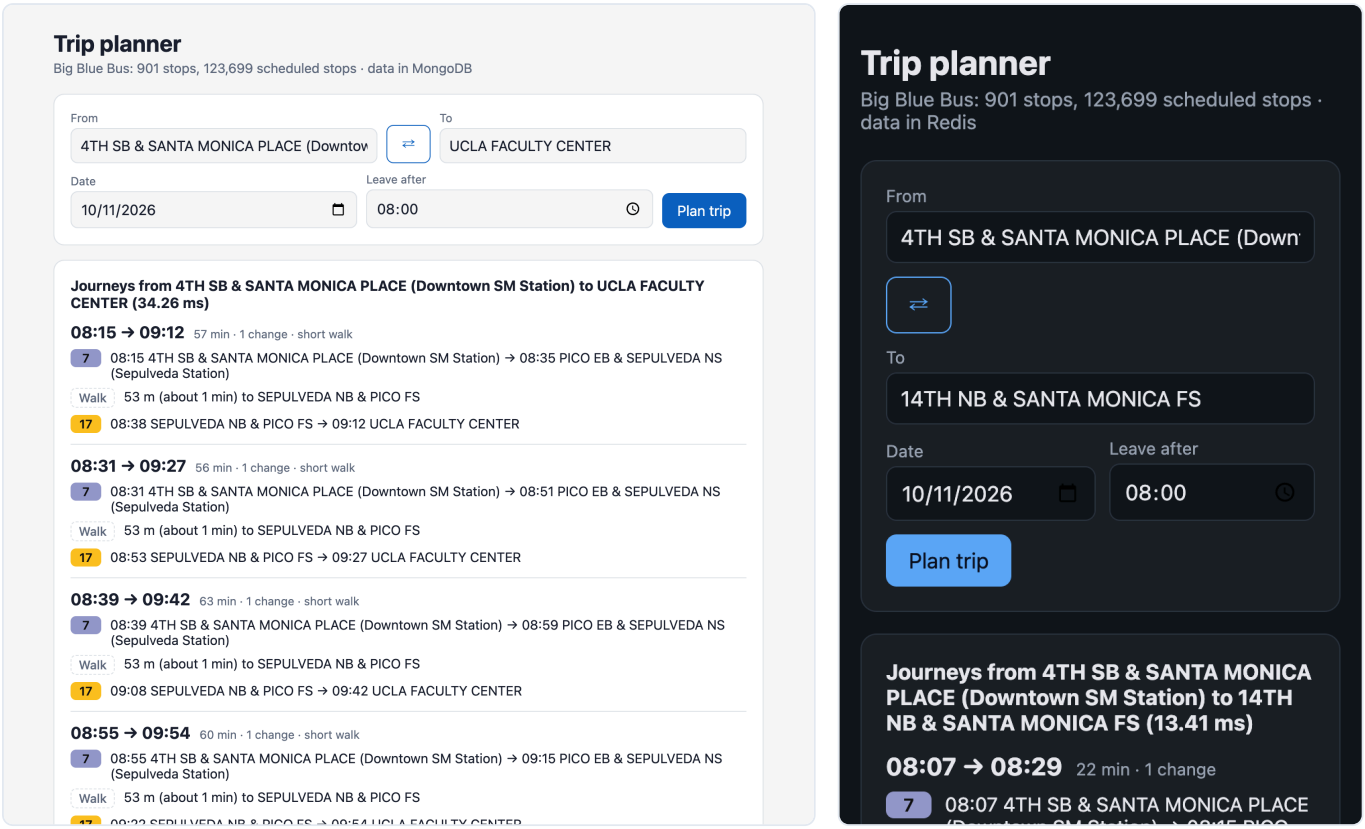
Scaling beyond one machine

- **Redis** scales reads with replicas and data with **Redis Cluster**, which splits keys into 16,384 hash slots across nodes. Commands that touch several keys must keep them in the same slot (hash tags like `{stop:13248}`), which constrains the key design. The real limit is **RAM**: TriMet alone needs 395 MB; a country with hundreds of feeds, plus several timetable versions and history, would need tens of gigabytes of RAM, which is expensive. Without AOF, a crash loses every write since the last snapshot.
- **MongoDB** scales reads with replica sets and data with **sharding**: for GTFS, a shard key like `{ feed_id, stop_id }` keeps one agency's stops together. Data lives on disk compressed, only hot data needs RAM, writes are durable, and new query patterns only need a new index.

My recommendation for a large-scale trip planner: MongoDB as the main database, with Redis as a cache. MongoDB holds the full timetable for all agencies (it is cheaper, durable, flexible and shards naturally by agency). Redis keeps the *hot* answers: departure boards of busy stops, recent plans, and live GTFS-Realtime delays, with an expiry time. Popular requests then get Redis speed, while the bulk of the data does not have to live in RAM. (Many production planners, such as OpenTripPlanner, go further and build an in-memory graph of the network inside the application; the database is then mainly for storage and the serving layer.)

7. Practical application: a simple trip planner

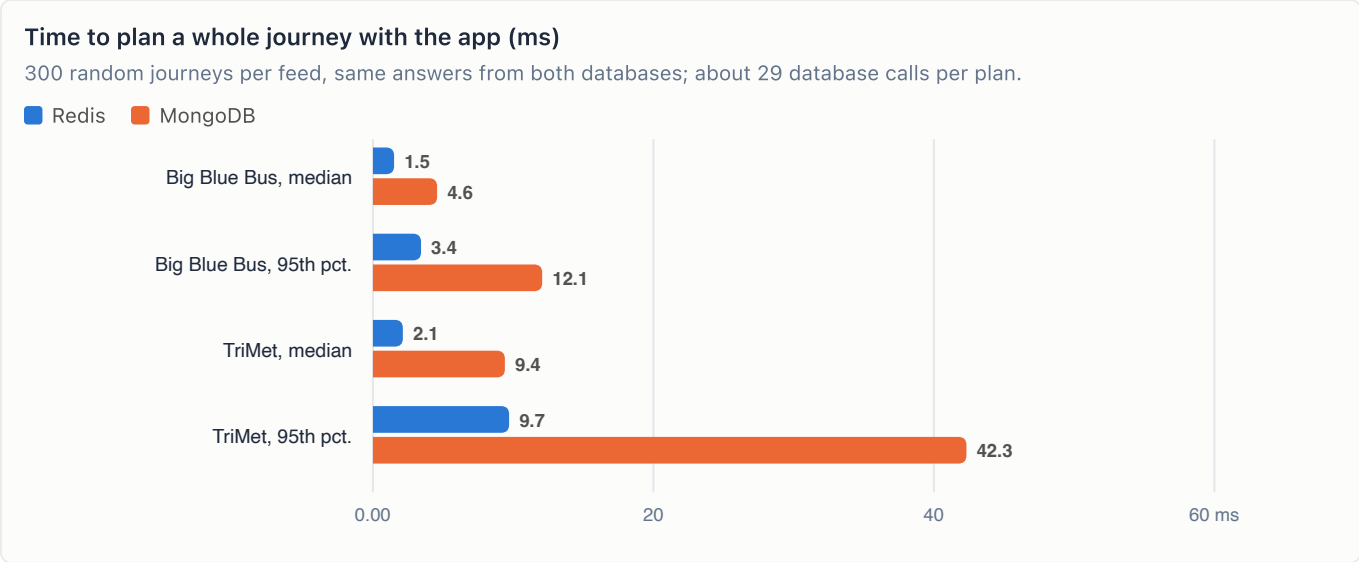
I built a small web app in `app/`, with Express and plain HTML/JavaScript. It runs on **either** database (`npm run app` or `DB=redis npm run app`), because the planner only calls the store methods I benchmarked. I made **MongoDB the default**, following my recommendation.



Top of the page only. Left: Downtown Santa Monica → UCLA Faculty Center on MongoDB: route 7, walk 53 m around the corner, route 17. Right: the same app on Redis, on a phone in dark mode. Route badges use the colours from `routes.txt`.

How I implemented it (step by step)

1. **Load the data:** download the GTFS ZIP (`npm run data`), parse it as a stream, load it into the database with the layout from section 4, and build the indexes.
2. **Service calendar:** for the chosen date, work out which services run (calendar + exceptions). This is tiny, so it stays in the app's memory.
3. **API** (Express): `/api/stops/search?q=` (autocomplete), `/api/stops/near?lat=&lon=`, `/api/departures/:stop?date=&time=`, `/api/plan?from=&to=&date=&time=`. It validates the input and answers 400 or 404 with a clear message.
4. **Planner:** (1) direct buses from A to B; (2) if there are fewer than 3: walk from A to a stop within 250 m, or take a bus to a stop where a second bus continues to B, changing at the same stop or a nearby one (the geo query). It allows at least 2 minutes to change (more if walking, at 1.2 m/s), keeps only journeys that no other journey beats (leaves later *and* arrives earlier), and never suggests walking to catch a bus that also stops at A.
5. **Front-end:** two stop pickers with autocomplete, a date and time, journey cards and a live departure board.
6. **Tests:** 20 automated tests on a handmade feed with known answers (loops, 25:10 times, a holiday that cancels the weekday service), run on **both** databases, plus API tests.



This is the most "application-based" result: because one plan makes about 29 small calls, the per-call difference adds up to **4.4× at the median and 4.4× at the 95th percentile on TriMet**. Still, 9.41 ms is fast for a web request. Both databases gave the same journeys for all 600 plans, except the walking distance shown (a metre of rounding) in 12 of them. Many random pairs had no journey (270 of 300 on TriMet), because my simple planner allows at most one change. A real planner would use an algorithm like RAPTOR for any number of changes.

8. Conclusion

	Redis	MongoDB
Advantages	• Fastest queries (2.5× on average here) and best tail latency • About 1.5× the throughput • Fastest loading • Sorted sets and geo commands fit timetables very well • Great for caching and real-time data (live delays, expiry)	• Data larger than RAM; 224 MB on disk vs 395 MB of RAM for TriMet • Durable by default; sharding and replica sets built in • Query language, server-side filtering, a planner that picks indexes • New questions need a new index, not a new data layout • Documents map naturally to GTFS rows
Disadvantages	• Everything must fit in (expensive) RAM • Every query designed by hand: the keys are the indexes (145 lines of code vs 126) • Filtering happens in the app: a wrong layout was 3× slower • Durability needs AOF; Cluster limits multi-key commands	• Slower per query (medians still under 1 ms here) • Many small calls per request add up (4.4× for a full plan) • Lower throughput and higher tail latency under heavy load • Uncompressed documents are big (field names)

Which would I choose for future GTFS projects? MongoDB as the main database, and Redis only when the measurements show I need it. In my benchmark MongoDB answered every trip-planning question correctly, with a median time under a millisecond, handled the 28× bigger network with little slowdown (except the departures layout I then fixed), and needed only 57% of the space Redis needed. It also made my life easier: I could add a new query with an index, and the planner adopted the better index on its own. Redis was clearly faster, and that speed matters for a busy service where one request makes dozens of calls. So for a large deployment I would add Redis as a cache for departure boards and live data, not as the only store, because keeping every agency's full timetable in RAM is costly and fragile.

Limits of this benchmark. Both databases and the client ran on one laptop (Apple M3 Pro, 18 GB RAM), so there was no network latency. In production, every call would also pay a network round trip, which would increase the gap for many-call requests. The data always fitted in RAM, which favours both; MongoDB's advantage when the data is larger than RAM was not tested. Each database ran as a single server with default settings, not a cluster. My data models are reasonable but not the only possible ones, as section 6 shows. Agencies publish new feeds every few months, so a re-run will give slightly different numbers.

Sources

- GTFS Schedule reference (gtfs.org): https://gtfs.org/documentation/schedule/reference/
- Big Blue Bus GTFS feed: https://gtfs.bigbluebus.com/current.zip · TriMet developer resources and GTFS: https://developer.trimet.org/
- Redis sorted sets: https://redis.io/docs/latest/develop/data-types/sorted-sets/ · geospatial: https://redis.io/docs/latest/develop/data-types/geospatial/
- Redis persistence (RDB and AOF): https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/ · scaling with Redis Cluster: https://redis.io/docs/latest/operate/oss_and_stack/management/scaling/
- MongoDB geospatial queries: https://www.mongodb.com/docs/manual/geospatial-queries/ · WiredTiger storage engine: https://www.mongodb.com/docs/manual/core/wiredtiger/ · sharding: https://www.mongodb.com/docs/manual/sharding/
- OpenTripPlanner: https://www.opentripplanner.org/

Measured with Redis 8.10.2 and MongoDB 8.2.6 (temporary local servers), Node v20.19.4, travel date Tuesday 10 November 2026. Code, raw results and tests: `gtfs-redis-mongodb-benchmark/` (`npm run bench`, `npm test`, `npm run app`).